28

BÖLÜM: DEPOLAMA, GÜVENLIK & OTOMATIK ÖLÇEKLEME | MODULE: STORAGE, SECURITY & HORIZONTAL AUTOSCALING

Pod Güvenlik Standartları: SecurityContext & Non-Root

Hardening Pods: SecurityContext & Non-Root Execution

💡 Aptala Anlatım (ELI5 Metaforu)

🇹🇷 TÜRKÇE

Konteyneri çelik bir zırh içine kilitlemek! Bir korsan Spring Boot uygulamanızın içine sızmayı başarsa bile elini kolunu bağlamak istiyoruz: 1) 'Root değilsin, garibansın!' (`runAsNonRoot`), 2) 'Sunucudaki hiçbir dosyayı değiştiremezsin!' (`readOnlyRootFilesystem`), 3) 'Hiçbir ayrıcalık talep edemezsin!' (`allowPrivilegeEscalation: false`). Korsan içeri girse bile hiçbir şey yapamaz!

💡 ELI5 Metaphor

🇬🇧 ENGLISH

Locking the container inside an iron cage! Even if a hacker breaches your Spring Boot app, you want them handcuffed: 1) 'You are not root; you are a powerless user!' (`runAsNonRoot`), 2) 'You cannot alter any system files!' (`readOnlyRootFilesystem`), 3) 'You cannot elevate permissions!' (`allowPrivilegeEscalation: false`). The hacker is trapped and helpless!

🔍 Derinlemesine Teknik Bakış

🇹🇷 TEKNİK DETAY

Pod Security Standards (PSS) üç düzey tanımlar: Privileged, Baseline ve Restricted. Üretim ortamında 'Restricted' hedeflenmelidir. `securityContext` bloğunda: 1) `runAsUser: 10001` (Root olmayan UID), 2) `allowPrivilegeEscalation: false` (setuid ikili dosyalarıyla yetki yükseltmeyi engeller), 3) `capabilities: drop: ['ALL']` (Linux çekirdek yeteneklerinin tamamını kaldırır), 4) `readOnlyRootFilesystem: true` (Konteynerin kök dosya sistemini salt-okunur yapar; yazılması gereken `/tmp` klasörü `emptyDir` ile ayrıca bağlanır).

🔍 Technical Deep-Dive

🇬🇧 TECHNICAL CORE

Pod Security Standards (PSS) define Privileged, Baseline, and Restricted profiles. Secure production pods must adhere to Restricted profiles: 1) `runAsNonRoot: true` with explicit UID (10001), 2) `allowPrivilegeEscalation: false`, 3) `capabilities.drop: ['ALL']` stripping Linux kernel capabilities, 4) `readOnlyRootFilesystem: true` with an ephemeral `emptyDir` mounted at `/tmp` for JVM temp writes.

☕ Java Spring Boot Pratiği

☕ Spring Boot in Action

```
# Zırhlı Pod Güvenlik Bloğu:
spec:
  securityContext:
    runAsNonRoot: true
    runAsUser: 10001
    runAsGroup: 10001
    fsGroup: 10001
    seccompProfile:
      type: RuntimeDefault
  containers:
    - name: coffee-app
      image: ghcr.io/eli5-k8s/spring-boot-k8s-eli5:1.0.0
      securityContext:
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
        capabilities:
          drop:
            - ALL
      volumeMounts:
        - mountPath: /tmp
          name: tmp-volume # Tomcat geçici dosyaları için geçici RAM diski
  volumes:
    - name: tmp-volume
      emptyDir: {}
```

⚠️ Acemi Tuzağı & Pro İpucu

⚠️ Gotchas & Pro-Tips

🇹🇷 AMAN DİKKAT

Spring Boot (özellikle gömülü Tomcat) `/tmp` klasörüne dosya yazmaya ihtiyaç duyar. Eğer `readOnlyRootFilesystem: true` yapıp `/tmp` klasörüne bir `emptyDir` bağlamazsanız uygulamanız açılışta `IOException` ile patlar!

🇬🇧 WATCH OUT

Embedded Tomcat crashes if it cannot write PID files or sockets to `/tmp`. When enabling `readOnlyRootFilesystem: true`, always mount an `emptyDir` to `/tmp`.

🎯 Özet Çıkarımlar

🎯 Key Takeaways

- Konteyneri root olmayan kullanıcıyla çalıştırmak container-breakout saldırılarını önler.
- `readOnlyRootFilesystem` zararlı yazılımların ikili dosya indirmesini engeller.
- Linux yeteneklerinin (capabilities) tamamı varsayılan olarak düşürülmelidir.

- Non-root execution prevents catastrophic container breakout host compromises.
- Read-only root filesystems block malware from persisting payloads to disk.
- Dropping ALL Linux capabilities minimizes OS kernel attack surfaces.